## Question #11 (Topic: Exam A) *Topic 1*

Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document analysis agent frequently identifies gaps – for instance, noting "the retrieved sources discuss API authentication but lack details on token refresh patterns" – but under the current strict pipeline, this insight isn't actionable since search has already completed. What's the most effective architectural change?

A. Add a research planning agent before the search phase that decomposes topics into specific sub-questions.

B. Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient. **Most Voted**

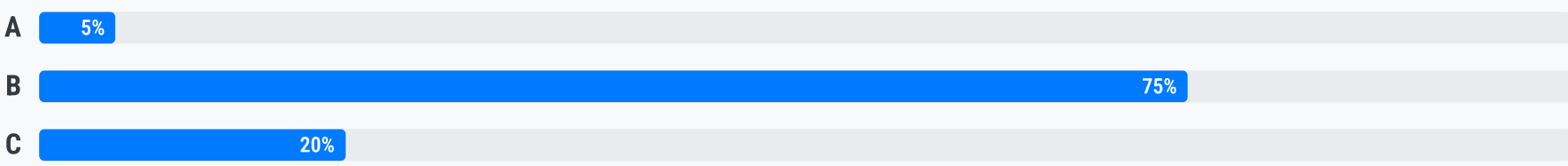
C. Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected.

D. Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review.

**Correct Answer: B**

Community vote currently supports Answer B

**Community vote distribution**




## Question #12 (Topic: Exam A) *Topic 1*

After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings. What is the correct approach for providing the synthesis subagent with the information it needs?

A. Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator

B. Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks

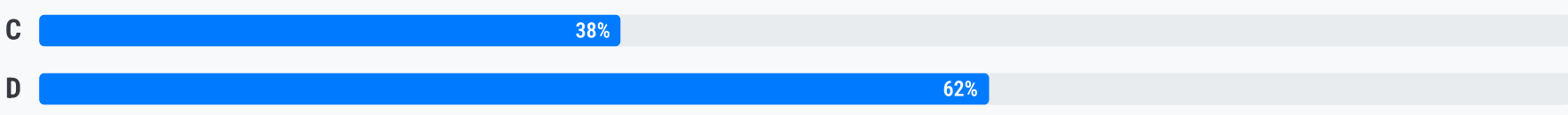
C. Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results

D. Include the complete findings from both subagents directly in the synthesis subagent's prompt **Most Voted**

**Correct Answer: D**

Community vote currently supports Answer D

**Community vote distribution**



**Question #13** (Topic: Exam A) *Topic 1*

When the agent calls lookup_order and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call process_refund or escalate_to_human next?

A. The orchestration layer automatically routes to the next tool based on the order's status field.

B. The order details are added to the conversation and the model reasons about which action to take. **Most Voted**

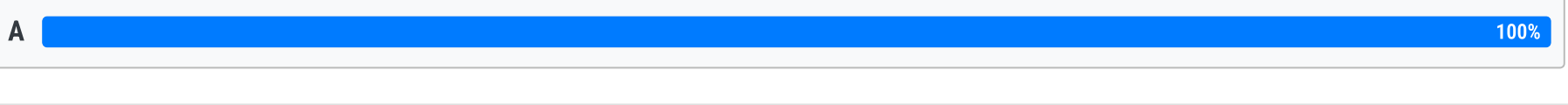
C. The agent executes the remaining steps in a tool sequence planned at the start of the request.

D. The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.

**Correct Answer: B**

Community vote currently supports Answer B

**Community vote distribution**




**Question #14** (Topic: Exam A) *Topic 1*

After investigating a billing dispute over 25+ turns, you've identified that duplicate charges occurred due to a payment gateway timeout triggering retry logic. The required refund ($847) exceeds your $500 authorization limit You need to call escalate_to_human, and the human agent won't have access to your conversation transcript. What context should you pass to enable effective resolution?

A. A structured summary: customer ID, root cause, refund amount, and recommended action. **Most Voted**

B. Your diagnosis and the refund amount only.

C. The customer's original complaint verbatim plus the tool result excerpts showing duplicate transactions.

D. The complete conversation transcript with all tool results.

**Correct Answer: A**

Community vote currently supports Answer A

**Community vote distribution**




**Question #15** (Topic: Exam A) *Topic 1*

Your agent is handling a billing dispute. After calling get_customer and lookup_order, it identifies that the dispute involves a promotional pricing error requiring manager approval – beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

A. Compile a structured handoff with customer details, order info, and the identified issue before calling escalate_to_human. **Most Voted**

B. Persist the complete conversation and tool response history to a database, then call escalate_to_human with a reference ID.

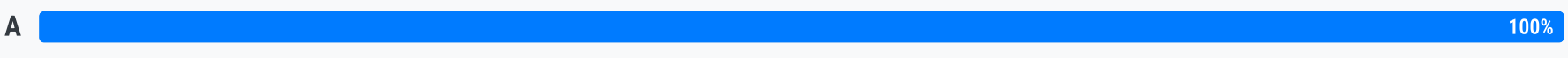
C. Call escalate_to_human passing only the customer's original message.

D. Attempt the refund with process_refund anyway, escalating only if the system rejects the transaction.

**Correct Answer: A**

Community vote currently supports Answer A

**Community vote distribution**



**Question #16** (Topic: Exam A) *Topic 1*

You're implementing the escalation logic for when the agent should call escalate_to_human. Your team proposes four different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

A. Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and trigger escalation when the frustration score exceeds a configured threshold.

B. Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.

C. Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.

D. Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls.

**Correct Answer: C** Community vote currently supports Answer

**Community vote distribution**

C 100%

**Question #17** (Topic: Exam A) *Topic 1*

Your order management system requires tools for three distinct operations: issuing refunds (requires amount and reason), canceling orders (requires reason), and requesting reshipments (requires shipping address). Each operation shares an order_id parameter but has different additional requirements. You notice during testing that with your current unified tool design, the agent frequently omits required parameters or includes irrelevant ones. What design change will most effectively improve parameter accuracy?

A. Keep one unified tool with a nested operation_details object parameter whose internal structure varies by operation type, documented in the tool description.

B. Keep one unified tool but add JSON Schema if-then-else conditionals to enforce that parameters like amount are required only when the operation type is "refund".

C. Split into three separate tools (issue_refund, cancel_order, request_reshipment), each defining only the parameters required for that specific operation. **Most Voted**

D. Keep one unified tool with all parameters marked optional, but add detailed few-shot examples in the system prompt showing correct parameter combinations for each operation type.

**Correct Answer: C** Community vote currently supports Answer C

**Community vote distribution**

C 100%

**Question #18** (Topic: Exam A) *Topic 1*

Your post_content tool requires user confirmation before publishing. The current workflow displays “Ready to post to social media. Confirm?” and analytics show users approve 98% of requests within 2 seconds. Post-mortems reveal incidents where posts went to wrong accounts, were scheduled for wrong times, or contained errors – all confirmed by users without catching the mistakes. How should you redesign the confirmation workflow?

A. Auto-approve routine posts and only require explicit confirmation for unusual patterns like posting to new accounts or large audiences

B. Require users to type a confirmation phrase instead of clicking a button

C. Add a mandatory waiting period before the confirm option becomes available

D. Include the complete post text, target account, scheduled time, and platform in the confirmation request

**Correct Answer: D** Community vote currently supports Answer

**Community vote distribution**




**Question #19** (Topic: Exam A) *Topic 1*

Your agent uses three tools: get_property_details(property_id) returns data including street address, get_price_history(property_id) returns historical pricing, and get_neighborhood_info(address) returns area statistics. You observe that get_neighborhood_info always requires get_property_details first just to extract the address, even when users specify the property by ID. This creates unnecessary latency and failure coupling – if the first call fails, the neighborhood request also fails. What tool design change best addresses this?

A. Create a lookup_address(property_id) helper tool for retrieving addresses.

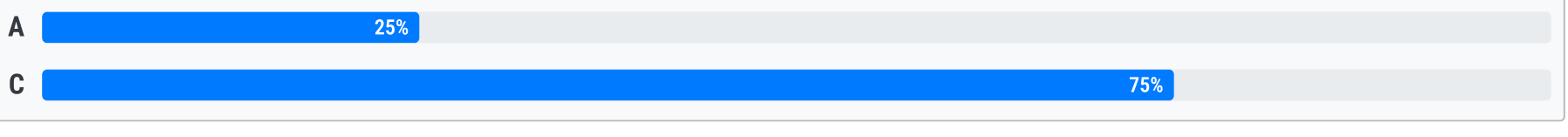
B. Add retry logic and timeout handling to get_property_details.

C. Change get_neighborhood_info to accept property_id, resolving the address internally. Most Voted

D. Consolidate into a single get_property_with_neighborhood(property_id) tool returning both datasets.

**Correct Answer: C** Community vote currently supports Answer C

**Community vote distribution**



**Question #20** (Topic: Exam A) *Topic 1*

Your update_user_profile tool accepts a user_id (required) and an optional fields_to_update object. In testing, Claude frequently omits user_id or passes incorrectly structured data. What is most critical for helping Claude understand what parameter values to provide?

A. Strict JSON Schema type constraints marking user_id as required and defining fields_to_update as an object type

B. Verbose parameter names encoding format hints, such as user_id_string_uuid_format

C. Detailed error responses explaining why invalid parameter values were rejected

D. Clear parameter descriptions explaining expected format, such as “user_id: UUID of the user to update (required)” **Most Voted**

**Correct Answer: D**

Community vote currently supports Answer D

**Community vote distribution**




Previous Questions

Next Questions

Browse atleast **50%** to increase passing rate

Viewing page 2 out of 14 pages.

Viewing questions **11-20** out of 134 questions